

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Computer Science and
Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Narravula Navya Sri	Reg. No.:	192324220
Section / Batch:	AI & DS	Date:	12-08-26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	An e-commerce system dynamically updates product prices and must frequently sort them for display. Analyze how Quick Sort can handle real-time sorting. Identify issues such as pivot selection and uneven data distribution. Apply divide and conquer techniques to explain the process. Analyze performance and propose optimizations with justification.	Q1

SECTION A – CASE STUDY

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				

Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

LanguagePythonC++Java

QUESTIONS

Q1
Dynamic Pricing Engine (Quick Sort)
Sort
100 marks
1/1 answered

ASSESSMENT C03/AT2 CASE STUDY

Dynamic Pricing Engine (Quick Sort)
100 marks

An e-commerce system dynamically updates product prices and must frequently sort them for display. Analyze how Quick Sort can handle real-time sorting. Identify issues such as pivot selection and uneven data distribution. Apply divide and conquer techniques to explain the process. Analyze performance and propose optimizations with justification.

Your Code

```
import sys
def quick_sort(arr, start, end):
    if start < end:
        pivot = arr[start]
        left = start
        right = end
        while left < right:
            while left < right and arr[left] < pivot:
                left += 1
            while left < right and arr[right] > pivot:
                right -= 1
            if left < right:
                arr[left], arr[right] = arr[right], arr[left]
            left += 1
            right -= 1
        arr[start], arr[right] = arr[right], arr[start]
        quick_sort(arr, start, left-1)
        quick_sort(arr, right+1, end)
    return arr

if __name__ == '__main__':
    arr = list(map(int, sys.stdin.readline().split()))
    n = len(arr)
    sorted_arr = quick_sort(arr, 0, n-1)
    print(' '.join(map(str, sorted_arr)))
```

Input
8
58 39 34 28 28 39 39 34

Output
28 28 34 34 39 39 39 58

Submit

Hot days ahead
32°C

Search

ENG IN

10:29
12-08-2026